

Evaluating RAG Pipelines with RAGAS

Naresh Chaurasia

Version 1.0, 2026-07-03: First Edition

Table of Contents

1. The Problem	1
1.1. When Good Looks Good But Is Not	1
2. The Concept	2
2.1. Grading a Pipeline the Way a Teacher Grades an Exam	2
3. How It Works	3
3.1. The Four-Metric Engine	3
3.2. Step-by-Step Flow of a RAGAS Evaluation Run	3
4. Key Components	5
4.1. Faithfulness	5
4.2. Answer Relevancy	5
4.3. Context Precision	6
4.4. Context Recall	6
4.5. The Evaluation Dataset	7
4.6. The Evaluation Harness	7
5. How It Solves the Problem	9
5.1. Closing the Loop on FreightBot	9
6. Summary	10
7. Quick Self-Check Questions	11
7.1. Foundational Questions	11
7.2. Intermediate Questions	11

Chapter 1. The Problem

1.1. When Good Looks Good But Is Not

Imagine you are a team of three engineers at a mid-sized logistics company. You have just shipped a Retrieval-Augmented Generation (RAG) chatbot called **FreightBot** that answers questions from a 400-page operations manual. After two weeks in production, customer-service agents start complaining. The bot gives answers that **sound** fluent and confident but are sometimes factually wrong, sometimes vague, and occasionally off-topic altogether.

Here is a sampler of real complaints logged in week two:

Question Asked	FreightBot Answer	What the Manual Says
"What is the cutoff time for same-day freight booking?"	"Freight bookings can be made during normal business hours."	"Same-day bookings must be submitted before 14:00 local time."
"Which carriers support temperature-controlled loads?"	"Several carriers in our network offer specialised services."	"ArcticFreight, PolarMove, and ChillChain support temperature-controlled loads."
"How do I file a damage claim?"	"You can contact our support team for assistance."	"Submit Form DA-7 within 48 hours via the Customer Portal."

The answers are grammatically correct. A casual reader might not notice anything wrong. But the agents, who need **precise** operational information, are getting burned several times a day.

You run a quick manual inspection of 50 question-answer pairs. Your findings:

- In 14 of 50 cases, the answer contained information that was **not** in the retrieved documents.
- In 18 of 50 cases, the answer was technically sourced from the documents but **did not actually answer** the question asked.
- In 9 of 50 cases, the retrieved documents were relevant but the answer missed key details present in those documents.
- In 7 of 50 cases, an irrelevant document was included in the retrieval context, crowding out the useful one.

The root problem is measurement. You shipped FreightBot with no systematic way to quantify **where** the pipeline is failing. Without measurement, every debugging session is guesswork. This is the problem that RAGAS was built to solve.

Chapter 2. The Concept

2.1. Grading a Pipeline the Way a Teacher Grades an Exam

Think of your RAG pipeline as a student sitting an open-book exam. The exam question is the user's query. The textbook is your document store. The student is allowed to look up relevant pages before answering.

A good examiner does not just mark the final answer right or wrong. The examiner asks four separate questions:

1. Did the student make up facts that are not in the textbook?
2. Did the student actually answer the question, or write around it?
3. Did the student look up only the relevant pages, or waste time on irrelevant ones?
4. Did the student find **all** the relevant pages, or miss some?

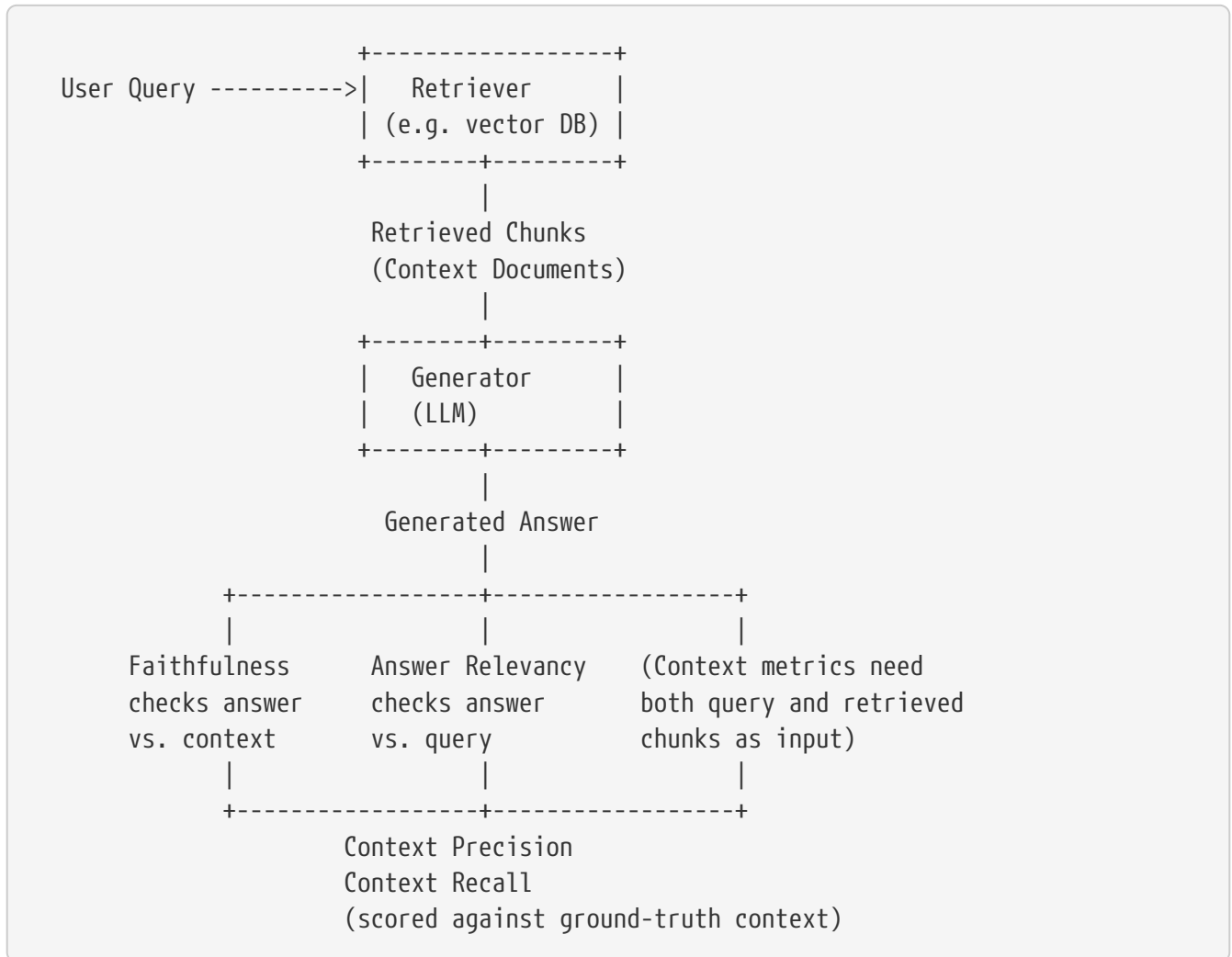
RAGAS — **Retrieval Augmented Generation Assessment** — applies exactly this four-part grading scheme to a RAG pipeline. Each question above maps to one metric. Together, the four metrics give you a diagnostic profile, not just a single score.

The word **automated** is important. RAGAS uses a language model internally to perform the grading. You do not need a human to score every answer; you prepare a reference dataset once, and RAGAS scores your pipeline on demand.

Chapter 3. How It Works

3.1. The Four-Metric Engine

Before diving into each metric individually, it helps to see the whole pipeline and where each metric attaches.



The retriever and generator are the two moving parts of your pipeline. Faithfulness and Answer Relevancy evaluate the **generator**. Context Precision and Context Recall evaluate the **retriever**.

3.2. Step-by-Step Flow of a RAGAS Evaluation Run

1. You prepare an **evaluation dataset** — a set of questions, each paired with a reference answer and the ground-truth context documents that should have been retrieved.
2. You run your RAG pipeline on those questions to collect, for each question: the retrieved context and the generated answer.
3. RAGAS takes (query, retrieved context, generated answer, reference answer, ground-truth context) as its inputs.
4. An internal LLM judge scores each metric independently.

5. RAGAS returns a score between 0 and 1 for each metric, for each sample, and computes dataset-level averages.

Applied to FreightBot, one row in your evaluation dataset might look like this:

Field	Value
query	"What is the cutoff time for same-day freight booking?"
ground_truth_context	"Same-day bookings must be submitted before 14:00 local time."
reference_answer	"The cutoff for same-day freight booking is 14:00 local time."
retrieved_context (pipeline output)	"Freight bookings can be made during normal business hours. Our team is available Monday to Friday."
generated_answer (pipeline output)	"Freight bookings can be made during normal business hours."

RAGAS then scores all four metrics on this single row.

Chapter 4. Key Components

4.1. Faithfulness

What it measures: Whether every factual claim in the generated answer can be traced back to something stated in the retrieved context.

Intuition: A faithful answer does not invent facts. If the retrieved text says "cutoff is 14:00" and the answer says "cutoff is 14:00", that claim is grounded. If the answer says "cutoff is end of business day" when the text says no such thing, that claim is hallucinated.

How RAGAS computes it:

1. Break the generated answer into atomic statements (individual claims).
2. For each claim, ask the internal judge: "Is this claim supported by the retrieved context?"
3. $\text{Score} = (\text{number of supported claims}) / (\text{total number of claims})$.

A score of 1.0 means every claim is grounded. A score of 0.5 means half the claims were invented.

FreightBot example: The answer "Freight bookings can be made during normal business hours" contains one claim. That claim is not directly supported by any precise statement in the retrieved chunk. Faithfulness score for this sample: approximately 0.3.

When faithfulness is low: The generator is hallucinating — either because the retrieved context is too thin, the prompt is not grounding the model firmly enough, or the LLM is over-confident.

4.2. Answer Relevancy

What it measures: Whether the generated answer directly addresses the question that was asked.

Intuition: A relevant answer is on-topic. "Contact our support team" is a real sentence, but it does not answer "How do I file a damage claim?" — it deflects.

How RAGAS computes it:

1. The internal judge generates several hypothetical questions that the given answer **would** be a good answer to.
2. It then measures the semantic similarity between those reverse-engineered questions and the original query.
3. $\text{Score} = \text{average cosine similarity between the original query and the generated back-questions}$.

A score of 1.0 means the answer speaks precisely to what was asked. A score close to 0 means the answer is off-topic or evasive.

FreightBot example: The answer "Several carriers in our network offer specialised services" does not name the carriers. A back-generated question from that answer might be "Do you offer

specialised freight services?" — far from the original "Which carriers support temperature-controlled loads?" Answer Relevancy score: approximately 0.35.

When answer relevancy is low: The generator is giving generic, hedge-filled, or tangential responses. Common causes: system prompt is too cautious, retrieved context is irrelevant, or the LLM is summarising instead of answering.

4.3. Context Precision

What it measures: Of all the chunks retrieved, what proportion are actually useful for answering the question?

Intuition: Imagine a research assistant who fetches ten pages from the manual to answer a single question. If seven of those pages are about unrelated topics and only three are relevant, the assistant has poor precision. Noisy context confuses the generator and dilutes the signal.

How RAGAS computes it:

1. For each retrieved chunk, the judge determines whether that chunk is relevant to the question (using the ground-truth context as reference).
2. Precision is computed with a ranking-aware formula that rewards pipelines which put the most relevant chunks first.
3. Higher-ranked relevant chunks contribute more to the score than lower-ranked ones.

A score of 1.0 means every retrieved chunk was useful and the most useful ones appeared at the top.

FreightBot example: The retriever fetched two chunks for the damage claim question. Chunk 1 was about general customer service contact details (irrelevant). Chunk 2 contained Form DA-7 instructions (relevant). Because the irrelevant chunk was ranked first, Context Precision is penalised: approximately 0.4.

When context precision is low: The retriever is fetching irrelevant documents. Causes include: weak embeddings, chunk size too large, similarity threshold too permissive, or index quality issues.

4.4. Context Recall

What it measures: Of all the information that was **needed** to answer the question correctly, how much did the retriever actually find?

Intuition: If the correct answer requires three facts and the retriever only surfaced one, the generator cannot possibly produce a complete answer even if it tries. Context recall measures coverage.

How RAGAS computes it:

1. The reference answer is broken into individual factual claims.
2. For each claim, the judge asks: "Can this claim be attributed to any of the retrieved chunks?"
3. $\text{Score} = (\text{claims attributable to retrieved context}) / (\text{total claims in reference answer})$.

A score of 1.0 means the retriever surfaced everything the reference answer requires.

FreightBot example: The reference answer for the temperature-controlled carrier question contains three carrier names: ArcticFreight, PolarMove, and ChillChain. The retriever fetched only the paragraph mentioning ArcticFreight and PolarMove. The ChillChain paragraph was in a different section that the retriever missed. Context Recall: $2 / 3 = 0.67$.

When context recall is low: The retriever is missing relevant documents. Causes include: chunking strategy that splits related content, poor embedding coverage, or a query that does not semantically match the relevant section headers.

4.5. The Evaluation Dataset

The evaluation dataset is the foundation that makes automated scoring possible. Without it, RAGAS has nothing to compare against.

A **golden question set** is a carefully curated collection of questions where humans have already determined:

- What the correct answer should be (the reference answer).
- Which document chunks should have been retrieved (the ground-truth context).

Guidelines for building a good golden set:

- Cover a variety of question types: factual lookup, multi-hop reasoning, comparative questions.
- Include at least one question per major topic area in your document corpus.
- Have subject-matter experts write or validate both the reference answers and the ground-truth context selections.
- Aim for a minimum of 50 to 100 samples for meaningful statistical averages; larger sets reduce variance.
- Refresh the golden set whenever the underlying document corpus changes significantly.

For FreightBot, the team built a golden set of 80 questions covering booking, claims, carrier selection, compliance, and escalation procedures.

4.6. The Evaluation Harness

The evaluation harness is the scaffolding that wires your pipeline to RAGAS. It consists of three components working together:

Pipeline runner: Takes each question from the golden set, runs it through your live RAG pipeline, and captures both the retrieved context and the generated answer. This runner must be deterministic or averaged over multiple runs if your pipeline is stochastic.

Dataset builder: Assembles the five-field table (query, retrieved context, generated answer, reference answer, ground-truth context) into a format RAGAS can consume. RAGAS expects a Hugging Face Dataset object or equivalent structure.

RAGAS scorer: Invokes the RAGAS evaluation function, which internally calls an LLM judge for each metric and aggregates scores across all samples. The scorer returns a results table and dataset-level averages.

The harness is typically run as part of a continuous integration pipeline — triggered whenever the retriever configuration, the embedding model, the chunking strategy, or the LLM prompt changes.

Chapter 5. How It Solves the Problem

5.1. Closing the Loop on FreightBot

Recall the four problem categories identified in the manual audit of FreightBot. Here is how each RAGAS metric maps directly to one of those categories and what the team did in response.

Observed Problem	RAGAS Metric	Score Found	Remediation Taken
Answers contained invented facts not in the documents	Faithfulness	0.41	Added a stricter system prompt instruction: "Answer only using the provided context. If the context does not contain the answer, say so explicitly."
Answers were technically sourced from docs but did not address the question	Answer Relevancy	0.38	Rewrote the prompt to require a direct answer to the specific question before elaborating.
Irrelevant chunks ranked first, confusing the generator	Context Precision	0.44	Reduced the number of retrieved chunks from 5 to 3 and raised the similarity threshold from 0.6 to 0.75.
Retriever missed entire sections of the manual	Context Recall	0.62	Reduced chunk size from 800 tokens to 400 tokens and added section-header overlap to each chunk.

After remediation, the team re-ran RAGAS on the full 80-question golden set.

Metric	Before	After
Faithfulness	0.41	0.81
Answer Relevancy	0.38	0.79
Context Precision	0.44	0.83
Context Recall	0.62	0.78

All four metrics improved materially. More importantly, the team now had **evidence** of improvement rather than anecdotal feedback. When the next sprint changed the embedding model, they re-ran RAGAS in under ten minutes and confirmed that Context Recall held steady.

This is the real value of an evaluation harness: it turns RAG improvement from art into engineering.

Chapter 6. Summary

A RAG pipeline has two places where things go wrong: the retriever and the generator. RAGAS provides four metrics — Faithfulness, Answer Relevancy, Context Precision, and Context Recall — that independently diagnose each failure mode. Faithfulness and Answer Relevancy tell you whether the generator is trustworthy and on-topic. Context Precision and Context Recall tell you whether the retriever is fetching the right material without noise and without gaps. No single metric is sufficient on its own; a pipeline can score well on three and catastrophically on the fourth, masking a serious flaw.

The evaluation harness — the golden question set, the reference answers, and the automated scoring loop — is the instrument that makes these metrics actionable. FreightBot’s team could have continued patching the pipeline based on user complaints, making slow and unreliable progress. Instead, RAGAS gave them a diagnostic readout that told them precisely **what** was broken and a repeatable test to confirm that their fix actually worked. Building this harness early, before user complaints accumulate, is one of the highest-leverage investments a team can make in a production RAG system.

Chapter 7. Quick Self-Check Questions

7.1. Foundational Questions

1. What does RAGAS stand for, and what is its general purpose?

Hint: Think about what the acronym expands to and what gap it fills for teams deploying RAG systems in production.

2. Which two RAGAS metrics evaluate the generator, and which two evaluate the retriever?

Hint: Consider where in the pipeline each metric gathers its evidence — from the answer alone, from the context alone, or from a comparison of the two.

3. In plain language, what does a Faithfulness score of 0.5 tell you about a generated answer?

Hint: Faithfulness is a ratio. Think about what the numerator and denominator represent.

4. What is a golden question set, and why must it be prepared before running RAGAS?

Hint: RAGAS needs something to compare the pipeline's output against. What provides that reference point?

5. If your Context Recall score is 0.6, what does that tell you the retriever is missing?

Hint: Recall is about coverage. Think about what fraction of the necessary information is not reaching the generator.

7.2. Intermediate Questions

1. FreightBot's Answer Relevancy score is low (0.38) but its Faithfulness score is high (0.85). What does this combination suggest about the pipeline's behaviour?

Hint: High faithfulness means the generator is not making things up. Low answer relevancy means something else is going wrong. What could cause an honest but unhelpful answer?

2. A team improves Context Precision from 0.44 to 0.83 by raising the similarity threshold for retrieval. What trade-off might this improvement introduce, and which metric would reveal it?

Hint: Fetching fewer, more precise chunks reduces noise — but what might happen to coverage? Which metric measures coverage?

3. Why does RAGAS weight earlier-retrieved chunks more heavily when computing Context Precision?

Hint: Think about how the generator uses its context. Does it pay equal attention to chunk 1 and chunk 10?

4. Your team changes the chunking strategy from 800-token chunks to 400-token chunks.

Which two RAGAS metrics are most likely to change, and in which direction would you expect them to move?

Hint: Smaller chunks affect both how much content fits in a chunk and how many related pieces of a topic end up in separate chunks. Think about retrieval noise versus retrieval coverage.

- 5. Why is it insufficient to rely on a single composite RAGAS score rather than examining all four metrics individually?**

Hint: Consider a scenario where three metrics are excellent and one is very poor. What does the average hide, and why does that matter for diagnosis?